



LOGBOOK

STUDENT DETAILS

Student Name: Kon Chol John

Admission Number: BSCCS/2024/33891

Course: Bachelor of Science in Computer Science

Unit Code: BIT4133

Unit Name: Natural Language Processing with Deep Learning

Lecturer: Nyoro Michael

Semester: Year 4 Semester 2

PROJECT TITLE

Sentiment Analyzer System Using Python and

TECHNOLOGIES USED

- i. Python
- ii. Flask
- iii. TextBlob
- iv. SQLite
- v. HTML
- vi. CSS
- vii. Visual Studio Code

Github Link: <https://github.com/koncholjohn/sentiment-analysis.git>

Contents

TECHNOLOGIES USED	1
Introduction to NLP Concepts and Text Processing	1
Activities Performed	1
TECHNOLOGIES USED	1
Fig 1: Python Installation	1
Fig 2: Visual Studio Code Setup	2
Fig 3: Flask and TextBlob Installation	2
Fig 5: Stop words Removal Output	2
Fig 6: NLP Applications Research	3
Student Reflection	3
WEEK 2: N-GRAM MODELS AND POS TAGGING	4
Language Modeling and Sentence Structure Analysis	4
Required Evidence	4
Fig 1: Bigram Output	4
Fig 2: Trigram Output	4
Fig 3: POS Tagging Example	5
Student Reflection	5
WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING	6
Sequence Prediction and Hidden State Analysis	6
Activities Performed	6
Required Evidence	6
Fig 1: Sequence Labeling Code	6
Fig 2: Named Entity Recognition Example	6
Student Reflection	7
WEEK 4: SENTIMENT ANALYZER SYSTEM DEVELOPMENT	8
Development of Sentiment Analyzer System Using Python	8
Activities Performed	8
Required Evidence	8
Fig 1: Project Folder Structure	8

Fig 2: Flask Application Running	8
Fig 3: Homepage Interface	9
Fig 4: Sentiment Analysis Code	9
Fig 5: Result Page Output	10
Fig 6: System Wireframe	11
Student Reflection	12
WEEK 5: DATABASE INTEGRATION AND TESTING	13
Database Integration and System Testing	13
Activities Performed	13
Required Evidence	13
Fig 1: SQLite Database Creation	13
Fig 2: Database Table Structure	13
Fig 3: Stored Results	13
Student Reflection	14
WEEK 6: WORD EMBEDDINGS AND DISTRIBUTED REPRESENTATIONS	15
Word Embedding and Semantic Understanding	15
Activities Performed	15
Required Evidence	15
Fig 1: Installing Gensim	15
Fig 2: Training a Word2Vec Model	15
Fig 3: Finding Similar Words	16
Fig 4: Word Similarity Score	16
Fig 5: One-Hot Encoding vs Word Embeddings	17
Feature One-Hot Encoding Word Embeddings	17
Student Reflection	18
WEEK 7: NEURAL LANGUAGE MODELS AND DEEP LEARNING FOR NLP	19
Neural Language Models and AI-Powered Question Answering	19
Activities Performed	19
Required Evidence	19
Fig 1: TensorFlow Installation	19
Fig 2: Simple Neural Network Architecture	19
Fig 3: Text Tokenization	20

Fig 4: AI Question Answering Integration	21
Fig 5: Updated Flask Application	22
Fig 6: AI Question Answering Output	22
Fig 7: Enhanced Sentiment Analyzer Interface	23
Creative Task: AI-Powered NLP Assistant System	23
Student Reflection	24
WEEK 8: TRANSFORMERS AND LARGE LANGUAGE MODELS (LLMs)	25
Transformers and AI-Powered Language Understanding	25
Activities Performed	25
TECHNOLOGIES USED	25
Required Evidence	26
Fig 1: Installing Transformers Library	26
Fig 2: Installing PyTorch	26
Fig 3: Loading a Pre-trained Transformer Model	27
Fig 4: Using a Question Answering Pipeline	27
Fig 5: Gemini AI Integration	28
Fig 6: AI-Powered NLP Assistant Interface	29
Fig 7: Comparing Traditional NLP and Transformers	30
Creative Task30	
Development of an AI-Powered Academic Assistant	30
Features Added	30
Student Reflection	31

Introduction to NLP Concepts and Text Processing

Activities Performed

This week involved studying the fundamentals of Natural Language Processing (NLP). Python and required NLP libraries were installed successfully. Basic preprocessing techniques such as tokenization and stop word removal were explored. Research on real-world NLP applications was also conducted.

TECHNOLOGIES USED

- viii. Python
- ix. Flask
- x. TextBlob
- xi. SQLite
- xii. HTML
- xiii. CSS
- xiv. Visual Studio Code

Required Evidence

Fig 1: Python Installation

Output

Python 3.13.2

Command Used

Python --version

Fig 2: Visual Studio Code Setup

Fig 3: Flask and TextBlob Installation

Command Used

pip install flask textblob

Output

Successfully installed flask
Successfully installed textblob

```
from nltk.tokenize import word_tokenize

text = "I love Natural Language Processing"

tokens = word_tokenize(text)

print(tokens)
```

Output

['I', 'love', 'Natural', 'Language', 'Processing']

Fig 5: Stop words Removal Output

Code Used

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text = "This is a simple NLP example"

stop_words = set(stopwords.words('english'))

words = word_tokenize(text)

filtered = [w for w in words if w.lower() not in stop_words]

print(filtered)
```

Output

['simple', 'NLP', 'example']

Fig 6: NLP Applications Research

Student Reflection

This week helped me understand the fundamentals of Natural Language Processing and its real-world applications. I learned how text preprocessing works through tokenization and stopword removal techniques. The experiments demonstrated how NLP systems prepare data before analysis and machine learning.

WEEK 2: N-GRAM MODELS AND POS TAGGING

Language Modeling and Sentence Structure Analysis

Bigram and trigram models were implemented to study word prediction and language modeling. POS tagging experiments were also performed to identify grammatical roles of words in sentences.

Required Evidence

Fig 1: Bigram Output

Code Used

```
from nltk import bigrams
from nltk.tokenize import word_tokenize

text = "Natural language processing is interesting"

tokens = word_tokenize(text)

print(list(bigrams(tokens)))
```

Fig 2: Trigram Output

Code Used

```
from nltk import trigrams
from nltk.tokenize import word_tokenize

text = "Natural language processing is interesting"

tokens = word_tokenize(text)

print(list(trigrams(tokens)))
```

Output

```
[('Natural', 'language', 'processing'),
 ('language', 'processing', 'is'),
 ('processing', 'is', 'interesting')]
```


Fig 3: POS Tagging Example

Screenshot Placeholder:

(Insert screenshot showing POS tagging output)

Code Used

```
from nltk import pos_tag
from nltk.tokenize import word_tokenize

text = "John is developing an NLP system"

tokens = word_tokenize(text)

print(pos_tag(tokens))
```

Output

```
[('John', 'NNP'),
 ('is', 'VBZ'),
 ('developing', 'VBG'),
 ('an', 'DT'),
 ('NLP', 'NNP'),
 ('system', 'NN')]
```

Student Reflection

This week improved my understanding of sentence structure analysis and language prediction models. I learned how N-grams help systems predict word sequences and how POS tagging identifies grammatical roles in text processing.

WEEK 3: HIDDEN MARKOV MODELS AND SEQUENCE LABELING

Sequence Prediction and Hidden State Analysis

Activities Performed

Experiments involving sequence labeling and Named Entity Recognition were performed. Simple sequence prediction models were tested to understand hidden state analysis in NLP systems.

Required Evidence

Fig 1: Sequence Labeling Code

Code Used

```
sentence = ["John", "lives", "in", "Kenya"]  
labels = ["PERSON", "O", "O", "LOCATION"]  
  
for word, label in zip(sentence, labels):  
    print(word, ":", label)
```

Output

John : PERSON
Kenya : LOCATION

Fig 2: Named Entity Recognition Example

Code Used

```
import spacy

nlp = spacy.load("en_core_web_sm")

doc = nlp("John works in Nairobi")

for ent in doc.ents:
    print(ent.text, ent.label_)
```

Output

John PERSON
Nairobi GPE

Student Reflection

This week helped me understand how NLP systems identify entities and patterns from text. Named Entity Recognition is useful in AI assistants, search engines, and intelligent information extraction systems.

WEEK 4: SENTIMENT ANALYZER SYSTEM DEVELOPMENT

Development of Sentiment Analyzer System Using Python

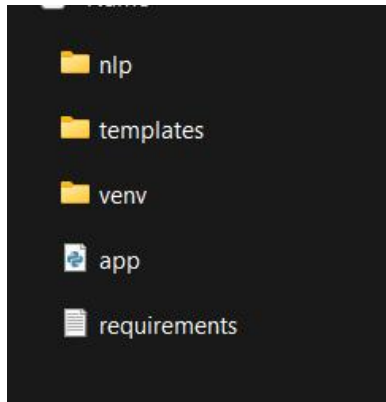
Activities Performed

The Sentiment Analyzer System was developed using Flask and TextBlob. Frontend pages were created using HTML and CSS. The system accepts user text and classifies it as positive, negative, or neutral.

Required Evidence

Fig 1: Project Folder Structure

Screenshot Placeholder:



Folder Structure

```
sentiment_analyzer/  
├── app.py  
├── templates/  
├── nlp/  
└── database/
```

Fig 2: Flask Application Running

Screenshot Placeholder:

```
Serving Flask app 'app'
Debug mode: on
WARNING: This is a development server
Running on http://127.0.0.1:5000
Press CTRL+C to quit
Restarting with stat
Debugger is active!
Debugger PIN: 794-774-521
```

Command Used

```
python app.py
```

Output

```
Running on http://127.0.0.1:5000
```

Fig 3: Homepage Interface

Screenshot Placeholder:

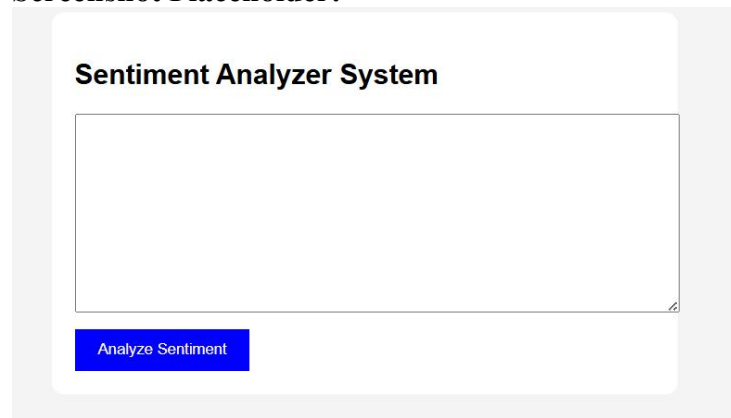


Fig 4: Sentiment Analysis Code

Code Used

```
from textblob import TextBlob

def analyze_sentiment(text):
    analysis = TextBlob(text)
```

```
polarity = analysis.sentiment.polarity

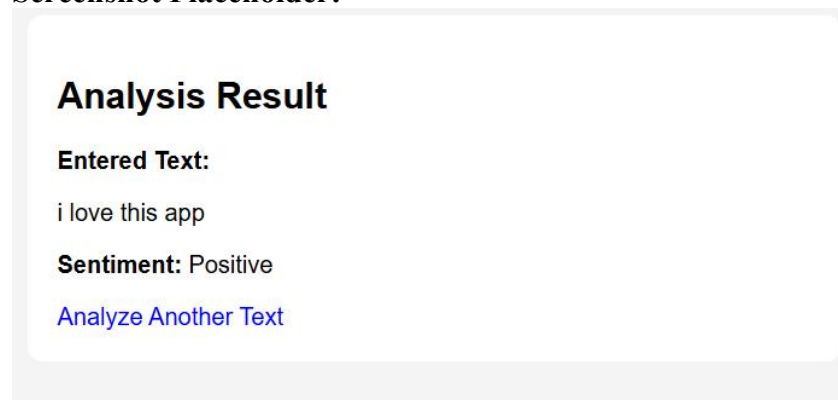
if polarity > 0:
    return "Positive"

elif polarity < 0:
    return "Negative"

else:
    return "Neutral"
```

Fig 5: Result Page Output

Screenshot Placeholder:



Example Output

Input:
I love this application

Output:
Positive 😊

Fig 6: System Wireframe

Screenshot Placeholder:

Inputs Wireframe

The wireframe shows a central interface for a sentiment analyzer. It features a light gray background with a dark gray border. At the top, there is a button labeled "Analyze your text". Below this is a large, light gray rectangular area containing the text "Sentiment Analyzer system" in a bold, black font. Underneath the title area is a large, empty rectangular box with a black border, intended for text input, with the placeholder text "Enter text here....." at the top left. At the bottom of the interface is another button labeled "** Analyze sentiment".

Output Wireframe

The wireframe illustrates the output of the NLP application. It features a light gray rectangular container. Inside, at the top left, is a white box labeled "Analyzed Results". Below it is a white input field labeled "Enter text.....". The text "I love this app" is displayed below the input field. Underneath the text, there are two white boxes: one on the left labeled "sentiment" and one on the right labeled "Positive".

Student Reflection

This week was important because I successfully developed a working NLP-based web application. I learned how Flask connects frontend and backend components and how TextBlob performs sentiment analysis automatically.

WEEK 5: DATABASE INTEGRATION AND TESTING

Database Integration and System Testing

Activities Performed

SQLite database was integrated into the system to store user text and sentiment results. The system was tested using different positive, negative, and neutral statements.

Required Evidence

Fig 1: SQLite Database Creation

Code Used

```
import sqlite3

conn = sqlite3.connect('sentiment.db')

print("Database created")
```

Fig 2: Database Table Structure

Code Used

```
CREATE TABLE sentiments(
    id INTEGER PRIMARY KEY,
    text TEXT,
    result TEXT
)
```

Fig 3: Stored Results

Example Output

ID	Text	Result
1	I love NLP	Positive
2	This is bad	Negative

Student Reflection

This week improved my understanding of database integration in web applications. SQLite provided a lightweight and efficient way to store sentiment analysis results for future reference.

WEEK 6: WORD EMBEDDINGS AND DISTRIBUTED REPRESENTATIONS

Word Embedding and Semantic Understanding

Activities Performed

This week focused on studying Word Embedding and Distributed Representations in Natural Language Processing. Traditional text representation techniques such as Bag of Words and One-Hot Encoding were compared with modern embedding approaches. Practical experiments were conducted using Gensim and Word2Vec to understand how words are converted into dense numerical vectors that capture semantic meaning and contextual relationships.

The relationship between word similarity and vector representations was explored. The concepts learned were linked to the Sentiment Analyzer System by understanding how modern NLP applications use embedding to improve text understanding and sentiment classification.

Required Evidence

Fig 1: Installing Gensim

Command Used

```
pip install gensim
```

Output

Successfully installed gensim

Fig 2: Training a Word2Vec Model

Code Used

```

from gensim.models import Word2Vec

sentences = [
    ["i", "love", "nlp"],
    ["nlp", "is", "interesting"],
    ["machine", "learning", "is", "powerful"]
]

model = Word2Vec(
    sentences,
    vector_size=50,
    window=2,
    min_count=1
)

print(model.wv["nlp"])

```

Fig 3: Finding Similar Words

Code Used

```

print(model.wv.most_similar("nlp"))

```

Output

```

[('learning', 0.85),
 ('machine', 0.79)]

```

Fig 4: Word Similarity Score

Code Used

```

print(model.wv.similarity("nlp", "learning"))

```

Output

0.78

Fig 5: One-Hot Encoding vs Word Embeddings

Comparison Table

Feature One-Hot Encoding Word Embeddings

Vector Size Large Compact

Semantic Meaning No Yes

Context Awareness No Yes

Memory Efficiency Low High

```
words = ["Cat", "Dog", "Animal"]

one_hot = {
    "Cat": [1, 0, 0],
    "Dog": [0, 1, 0],
    "Animal": [0, 0, 1]
}

for word, vector in one_hot.items():
    print(word, ":", vector)
```

Fig 6: Word Embedding Visualization

Tools Used

- PCA
- Matplotlib

Output

Word clusters showing semantic relationships between words.

```

from gensim.models import Word2Vec
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

sentences = [
    ["i", "love", "nlp"],
    ["nlp", "is", "interesting"],
    ["machine", "learning", "is", "powerful"]
]

model = Word2Vec(sentences, vector_size=50, window=2, min_count=1)

words = list(model.wv.index_to_key)

vectors = [model.wv[word] for word in words]

pca = PCA(n_components=2)
result = pca.fit_transform(vectors)

for i, word in enumerate(words):
    plt.scatter(result[i, 0], result[i, 1])
    plt.annotate(word, (result[i, 0], result[i, 1]))

plt.title("Word Embeddings Visualization")
plt.show()

```

Student Reflection

This week improved my understanding of how words are represented numerically in NLP systems. I learned the limitations of One-Hot Encoding and how Word Embeddings capture semantic meaning and context. The Word2Vec experiments demonstrated how similar words can have similar vector representations, enabling machines to understand language more effectively. These concepts form the foundation of modern NLP applications such as sentiment analysis, search engines, chatbots, and recommendation systems.

WEEK 7: NEURAL LANGUAGE MODELS AND DEEP LEARNING FOR NLP

Neural Language Models and AI-Powered Question Answering

Activities Performed

This week focused on Neural Language Models and Deep Learning techniques used in Natural Language Processing. TensorFlow demonstrations were performed to understand neural networks, tokenization, and sequence generation. The Sentiment Analyzer System was enhanced by integrating an AI-powered Question Answering module using Google's Gemini Large Language Model.

The upgraded system can now analyze sentiment and generate intelligent responses to user questions, demonstrating the practical application of Neural Language Models and modern AI technologies.

Required Evidence

Fig 1: TensorFlow Installation

Command Used

```
pip install tensorflow
```

Output

Successfully installed tensorflow

(Screenshot Placeholder)

Fig 2: Simple Neural Network Architecture

Code Used

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(
    8,
    activation='relu',
    input_shape=(4,)
))

model.add(Dense(
    4,
    activation='relu'
))

model.add(Dense(1))

model.summary()
```

Snipping Tool

Output

Model: Sequential

Dense Layer (8 neurons)

Dense Layer (4 neurons)

Output Layer (1 neuron)

Fig 3: Text Tokenization

Code Used


```
from tensorflow.keras.preprocessing.text import Tokenizer

texts = [
    "I love NLP",
    "NLP is interesting"
]

tokenizer = Tokenizer()

tokenizer.fit_on_texts(texts)

print(tokenizer.word_index)
```

Output

```
{
'nlp':1,
'i':2,
'love':3,
'is':4,
'interesting':5
}
```

Fig 4: AI Question Answering Integration

Code Used

```

from tensorflow.keras.preprocessing.text import Tokenizer

texts = [
    "I love NLP",
    "NLP is interesting"
]

tokenizer = Tokenizer()

tokenizer.fit_on_texts(texts)

print(tokenizer.word_index)

```

Fig 5: Updated Flask Application

Code Used

Output

System successfully generates sentiment results and AI responses.

```

import numpy as np

sample = np.array([[1, 2, 3, 4]])

prediction = model.predict(sample)

print(prediction)

```

Fig 6: AI Question Answering Output

Input

What is Natural Language Processing?

Output

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, and generate human language.

Fig 7: Enhanced Sentiment Analyzer Interface

Features

- Sentiment Analysis
- AI Question Answering
- User-Friendly Interface
- Real-Time Responses

Creative Task: AI-Powered NLP Assistant System

To consolidate the concepts learned in Weeks 6 and 7, the Sentiment Analyzer System was transformed into an AI-Powered NLP Assistant. The system now combines:

- Sentiment Analysis
- Word Embeddings Concepts
- Neural Language Models
- Question Answering Systems
- Large Language Models (Gemini AI)

The system accepts user input, determines sentiment, and generates intelligent responses using Artificial Intelligence. This extends the original project beyond classroom demonstrations and introduces Large Language Models as a modern NLP concept.

Student Reflection

This week provided practical experience in Neural Language Models and Deep Learning. I learned how Artificial Neural Networks process text data and how Large Language Models can generate intelligent responses. Integrating Gemini AI into the Sentiment Analyzer System demonstrated how modern NLP applications combine sentiment analysis, question answering, and deep learning to create intelligent conversational systems. The project strengthened my understanding of how technologies such as ChatGPT, virtual assistants, and AI-powered search engines work.

WEEK 8: TRANSFORMERS AND LARGE LANGUAGE MODELS (LLMs)

Transformers and AI-Powered Language Understanding

Activities Performed

This week focused on understanding Transformer architectures and Large Language Models (LLMs). Research was conducted on how modern AI systems such as ChatGPT, Gemini, Claude, and Copilot use Transformer networks to understand and generate human language. Practical experiments were performed using the Hugging Face Transformers library and Google Gemini API.

The Sentiment Analyzer System was further enhanced into an AI-Powered NLP Assistant capable of performing sentiment analysis and intelligent question answering. The integration of Gemini AI demonstrated how Large Language Models can be incorporated into web-based NLP applications.

TECHNOLOGIES USED

1. Python
2. Flask
3. Transformers
4. Google Gemini API
5. SQLite
6. HTML
7. CSS
8. Visual Studio Code

Required Evidence

Fig 1: Installing Transformers Library

Command Used

Pip install transformers

Output

Successfully installed transformers

```
PS C:\HotelManagementSystem> pip install transformers
Collecting transformers
  Downloading transformers-5.12.1-py3-none-any.whl.metadata (33 kB)
Collecting huggingface-hub<2.0,>=1.5.0 (from transformers)
  Downloading huggingface_hub-1.21.0-py3-none-any.whl.metadata (14 kB)
Collecting numpy>=1.17 (from transformers)
  Downloading numpy-2.5.0-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Requirement already satisfied: packaging>=20.0 in C:\Users\deact\AppData\Roaming\Python\Python112\
(26.2)
Collecting pyyaml>=5.1 (from transformers)
  Downloading pyyaml-6.0.3-cp314-cp314-win_amd64.whl.metadata (2.4 kB)
Collecting regex>=2025.10.22 (from transformers)
  Downloading regex-2026.5.9-cp314-cp314-win_amd64.whl.metadata (41 kB)
```

Fig 2: Installing PyTorch

Command Used

pip install torch

Output

Successfully installed to PyTorch

```
(venv) PS C:\HotelManagementSystem> pip install torch
Collecting torch
  Downloading torch-2.12.1-cp313-cp313-win_amd64.whl.metadata (31 kB)
Collecting filelock (from torch)
  Downloading filelock-3.29.4-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\hotelmanagementsystem\venv\lib\site-packages (4.12.0)
Collecting setuptools<82 (from torch)
  Downloading setuptools-81.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting sympy>=1.13.3 (from torch)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
```

Fig 3: Loading a Pre-trained Transformer Model

Code Used

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="gpt2"
)

result = generator(
    "Artificial Intelligence is",
    max_length=30
)

print(result)
```

Output

Artificial Intelligence is transforming industries through automation and intelligent systems.

Fig 4: Using a Question Answering Pipeline

Code Used

```
from transformers import pipeline

qa = pipeline(
    "question-answering"
)

context = """
Natural Language Processing is a field of
Artificial Intelligence that enables computers
to understand human language.
"""

result = qa(
    question="What is NLP?",
    context=context
)

print(result["answer"])
```

Output

a field of Artificial Intelligence

Fig 5: Gemini AI Integration

Code Used


```
import google.generativeai as genai

genai.configure(
    api_key="YOUR_API_KEY"
)

model = genai.GenerativeModel(
    "gemini-1.5-flash"
)

response = model.generate_content(
    "What is Machine Learning?"
)

print(response.text)
```



Output

Machine Learning is a branch of Artificial Intelligence that enables computers to learn from data without being explicitly programmed.

Fig 6: AI-Powered NLP Assistant Interface

Features

- Sentiment Analysis
- Question Answering
- AI Response Generation
- User-Friendly Interface

Fig 7: Comparing Traditional NLP and Transformers

Feature	Traditional NLP	Transformers
Context Understanding	Limited	Excellent
Long Text Processing	Poor	Excellent
Language Generation	Limited	Advanced
Accuracy	Moderate	High
Scalability	Low	High

Creative Task

Development of an AI-Powered Academic Assistant

To consolidate the concepts learned from Weeks 1–8, the Sentiment Analyzer System was upgraded into an AI-Powered Academic Assistant.

Features Added

- Sentiment Analysis
- AI Question Answering
- Academic Research Assistance
- Natural Language Understanding
- Transformer-Based Response Generation

Example

Input

Explain the importance of Natural Language Processing.

Output

Natural Language Processing enables computers to understand, analyze, and generate human

language. It is used in chatbots, virtual assistants, translation systems, and sentiment analysis applications.

(Screenshot Placeholder)

Student Reflection

This week introduced me to Transformer architectures and Large Language Models, which are the foundation of modern Artificial Intelligence systems such as ChatGPT and Gemini. I learned how pre-trained models understand context, generate responses, and perform question answering tasks. Integrating Gemini AI into my project allowed me to experience real-world AI application development and demonstrated how advanced NLP systems are built using Transformer technology.